

AI Operating System Workshop

Day 1: Data Engineering & RAG Foundation

วิศวกรรมข้อมูลและพื้นฐาน RAG

X-Company

2026

ส่วนที่ 1: ภาพรวม RAG

Retrieval-Augmented Generation (RAG) คือสถาปัตยกรรม AI ที่รวมการดึงข้อมูล (Retrieval) เข้ากับการสร้างข้อความ (Generation) ของ LLM โดยแทนที่จะพึ่งพาความรู้จากพารามิเตอร์ของโมเดลเพียงอย่างเดียว RAG จะค้นหาบริบทที่เกี่ยวข้องจากฐานความรู้ภายนอกก่อน แล้วส่งให้ LLM สร้างคำตอบที่แม่นยำและอ้างอิงได้

ขั้นตอนการทำงาน:

- คำถามผู้ใช้ → Embedding → ค้นหา Vector → ดึง Chunks
- Chunks + คำถาม → Prompt LLM → สร้างคำตอบ

RAG vs Fine-tuning

เกณฑ์	RAG	Fine-tuning
อัปเดตความรู้	Real-time (อัปเดต Vector DB)	ต้องเทรนใหม่
ค่าใช้จ่าย	ต่ำ (ไม่ต้องใช้ GPU training)	สูง (GPU hours)
ความแม่นยำข้อมูลใหม่	สูง	ต่ำ (catastrophic forgetting)
ความโปร่งใส	อ้างอิงแหล่งข้อมูลได้	Black-box weights
เหมาะกับ	Q&A เอกสารที่เปลี่ยนแปลงบ่อย	ปรับ style/format การตอบ
Latency	สูงกว่า (มีขั้น retrieval)	ต่ำกว่า

เมื่อไหร่ควรใช้ RAG หรือ Fine-tuning

- ใช้ RAG: เมื่อข้อมูลเปลี่ยนแปลงบ่อย, ต้องการอ้างอิงแหล่งข้อมูล, ข้อมูลเป็นความลับ
- ใช้ Fine-tuning: เมื่อต้องการ response style เฉพาะ, คำศัพท์ domain พิเศษมาก
- ใช้ทั้งคู่: เพื่อผลลัพธ์ที่ดีที่สุดในองค์กร — fine-tune สำหรับรูปแบบ, RAG สำหรับข้อเท็จจริง

ส่วนที่ 2: การประมวลผลเอกสาร

การดึงข้อความจาก PDF/DOCX ด้วย PyMuPDF

```
import fitz  # PyMuPDF

def extract_pdf(path: str) -> str:
    doc = fitz.open(path)
    text = ''
    for page in doc:
        text += page.get_text('text')
    return text
```

```
# ดึงข้อความจาก DOCX
from docx import Document

def extract_docx(path: str) -> str:
    doc = Document(path)
    return '\n'.join([p.text for p in doc.paragraphs])

text = extract_pdf('document.pdf')
print(text[:500])
```

ความท้าทายของ NLP ภาษาไทย: การตัดคำ

ภาษาไทยไม่มีช่องว่างระหว่างคำ ทำให้การตัดคำเป็นขั้นตอนที่สำคัญและยากมาก ปัญหาที่พบบ่อย:

- ไม่มีตัวคันคำ — ต้องอนุมานขอบเขตของคำ
- การตัดคำที่คลุมเครือ: 'ตากลม' อาจหมายถึง 'ตา (กลม)' หรือ 'ตาก (ลม)'
- คำใหม่และคำทับศัพท์ที่ไม่อยู่ใน dictionary
- ความหมายเปลี่ยนตาม context

การตัดคำด้วย PyThaiNLP

```
from pythainlp.tokenize import word_tokenize

text = 'การประมวลผลภาษาธรรมชาติมีความสำคัญมาก'

# เครื่องมือเริ่มต้น: newmm
tokens_newmm = word_tokenize(text, engine='newmm')
print('newmm:', tokens_newmm)

# Longest match
tokens_longest = word_tokenize(text, engine='longest')
print('longest:', tokens_longest)

# attacut (deep learning)
tokens_attacut = word_tokenize(text, engine='attacut')
print('attacut:', tokens_attacut)
```

เปรียบเทียบเครื่องมือตัดคำภาษาไทย

เครื่องมือ	อัลกอริทึม	ความเร็ว	ความแม่นยำ	เหมาะกับ
newmm	Dict + MaxMatch	เร็ว	ดี	ทั่วไป
longest	Longest match	เร็ว	ปานกลาง	ข้อความง่าย
attacut	Deep Learning (CNN)	ปานกลาง	สูง	ข้อความซับซ้อน/ทางการ
icu	Unicode rules	เร็วมาก	ต่ำ	ตัดคำเบื้องต้น

ส่วนที่ 3: กลยุทธ์การแบ่ง Chunk

Fixed-Size Chunking

แบ่งข้อความเป็น chunk ขนาดเท่ากัน พร้อม overlap ง่ายและเร็ว

```
def fixed_chunk(text: str, size: int = 512, overlap: int = 50) -> list[str]:
    chunks = []
    start = 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start += size - overlap
    return chunks

chunks = fixed_chunk(text, size=512, overlap=50)

print(f'จำนวน chunks: {len(chunks)}')
```

Recursive Chunking

แบ่งตามขอบเขตธรรมชาติ (ย่อหน้า → ประโยค → คำ) จนถึงขนาดที่กำหนด

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    separators=['\n\n', '\n', '.', ' ', '']
)

chunks = splitter.split_text(text)
```

Semantic Chunking

ใช้ embedding similarity เพื่อแบ่งตามหัวข้อที่มีความหมาย

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_community.embeddings import OllamaEmbeddings

embeddings = OllamaEmbeddings(model='bge-m3')
chunker = SemanticChunker(embeddings,
    breakpoint_threshold_type='percentile')
chunks = chunker.split_text(text)
```

เปรียบเทียบกลยุทธ์การแบ่ง Chunk

กลยุทธ์	ข้อดี	ข้อเสีย	เหมาะกับ
Fixed-size	ง่าย เร็ว คัดเอาได้	อาจตัดกลางประโยค	เอกสารขนาดใหญ่ สม่ำเสมอ
Recursive	เคารพขอบเขต ธรรมชาติ	ช้ากว่าเล็กน้อย	RAG ทั่วไป
Semantic	chunk สอดคล้องกับ	ช้า ต้องใช้ embeddings	บทความวิจัย รายงาน

	หัวข้อ		
--	--------	--	--

ส่วนที่ 4: Embeddings และ Vector Database

Embeddings คือการแปลงข้อความ เป็น vector ที่จับความหมายได้ ข้อความที่มีความหมายใกล้เคียงกันจะ ให้ vector ที่ใกล้กัน ช่วยให้ค้นหาแบบ semantic ได้

เปรียบเทียบโมเดล Embedding

โมเดล	มิติ	ภาษา	ขนาด	เหมาะกับ
bge-m3	1024	100+	570MB	หลายภาษา รองรับไทย
nomi-embed-text	768	อังกฤษหลัก	274MB	เอกสารอังกฤษ เร็ว
multilingual-e5-large	1024	100+	560MB	ค้นหาข้ามภาษา

ติดตั้ง Qdrant ด้วย Docker

```
# เริ่มต้น Qdrant ด้วย Docker
docker run -d --name qdrant \
  -p 6333:6333 -p 6334:6334 \
  -v $(pwd)/qdrant_storage:/qdrant/storage \
  qdrant/qdrant:latest
```

การ Index เอกสาร

```
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct
import ollama

client = QdrantClient('localhost', port=6333)
client.create_collection(
    collection_name='workshop_docs',
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE)
)

for i, chunk in enumerate(chunks):
    embedding = ollama.embeddings(model='bge-m3', prompt=chunk)
    client.upsert('workshop_docs', points=[
        PointStruct(id=i, vector=embedding['embedding'],
                    payload={'text': chunk})
    ])
```

การค้นหา Semantic

```
def search(query: str, top_k: int = 5) -> list[str]:
    query_vec = ollama.embeddings(model='bge-m3', prompt=query) ['embedding']
    results = client.search(
        collection_name='workshop_docs',
        query_vector=query_vec,
        limit=top_k
    )
    return [r.payload['text'] for r in results]
```

ส่วนที่ 5: Hybrid Search

Hybrid Search รวม dense vector similarity (semantic) กับ sparse keyword matching (BM25) เพื่อให้ได้ผลการค้นหาที่ครบถ้วนกว่า

Dense vs BM25

วิธีการ	Algorithm	จุดแข็ง	จุดอ่อน
Dense (Vector)	Embedding similarity	เข้าใจความหมาย	พลาดค่าที่แน่นอน
BM25 (Sparse)	TF-IDF based	จับคีย์เวิร์ดได้แม่น	ไม่เข้าใจ semantic
Hybrid	Weighted combination	รวมข้อดีทั้งคู่	pipeline ซับซ้อนขึ้น

สูตร Hybrid Search (RRF)

```
# Reciprocal Rank Fusion
def rrf_score(rank: int, k: int = 60) -> float:
    return 1.0 / (k + rank)

def hybrid_search(dense_results, sparse_results, alpha=0.7):
    scores = {}
    for rank, doc in enumerate(dense_results):
        scores[doc.id] = alpha * rrf_score(rank)
    for rank, doc in enumerate(sparse_results):
        scores[doc.id] = scores.get(doc.id, 0) + (1-alpha) * rrf_score(rank)
    return sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

กลยุทธ์ Reranking

- Cross-encoder reranking: ใช้โมเดล (เช่น bge-reranker-v2-m3) ให้คะแนนคู่ query-document
- LLM-as-reranker: ให้ LLM จัดอันดับ passage ตามความเกี่ยวข้อง
- Cohere Rerank API: API เชิงพาณิชย์สำหรับ production

ส่วนที่ 6: RAG Pipeline พื้นฐาน

```
import ollama
from qdrant_client import QdrantClient

client = QdrantClient('localhost', port=6333)
```

```
def rag_pipeline(question: str) -> str:

    # 1. แปลง query เป็น embedding
    q_vec = ollama.embeddings(model='bge-m3', prompt=question)['embedding']

    # 2. ดึง chunks ที่เกี่ยวข้องมากที่สุด 5 อัน
    results = client.search(
        collection_name='workshop_docs',
        query_vector=q_vec, limit=5
    )
    context = '\n\n'.join([r.payload['text'] for r in results])

    # 3. สร้าง prompt
    prompt = f'บริบท:\n{context}\n\nคำถาม: {question}\nคำตอบ: '

    # 4. สร้างคำตอบ
    response = ollama.chat(
        model='llama3.2',
        messages=[{'role': 'user', 'content': prompt}]
    )
    return response['message']['content']

answer = rag_pipeline('เอกสารที่ต้องใช้ในการเบิกค่าใช้จ่ายมีอะไรบ้าง?')
print(answer)
```

ผลลัพธ์ที่คาดหวัง: คำตอบที่กระชับ อ้างอิงจาก context ที่ดึงมาเท่านั้น โมเดลจะอ้างอิงถึงส่วนที่เกี่ยวข้อง
แทนการ hallucinate

ส่วนที่ 7: แบบฝึกหัด

แบบฝึกหัดที่ 1: Document Ingestion Pipeline

งาน: สร้าง pipeline ดึงข้อความจาก PDF 5 ไฟล์ และเก็บ chunks ใน Qdrant

- Hint: ใช้ PyMuPDF ดึงข้อความ, RecursiveCharacterTextSplitter แบ่ง chunk
- Bonus: เพิ่ม metadata (ชื่อไฟล์, หน้า) ให้แต่ละ chunk

แบบฝึกหัดที่ 2: การประมวลผลข้อความภาษาไทย

งาน: เปรียบเทียบผลการตัดคำ newmm vs attacut บนคำบรรยายสินค้าภาษาไทย

- Hint: ใช้ pythainlp.tokenize.word_tokenize กับ engine parameter ต่างๆ

แบบฝึกหัดที่ 3: Hybrid Search

งาน: ใช้ RRF รวมผล dense และ BM25 แล้วเปรียบเทียบกับ dense เพียงอย่างเดียว

- Hint: ใช้ library rank_bm25 สำหรับ sparse retrieval

แบบฝึกหัดที่ 4: Reranking

งาน: เพิ่ม cross-encoder reranker ใน RAG pipeline และวัดการปรับปรุง NDCG

- Hint: sentence-transformers CrossEncoder กับ 'BAAI/bge-reranker-v2-m3'

แบบฝึกหัดที่ 5: RAG ครบวงจร

งาน: สร้าง Q&A chatbot บนเอกสาร policy ของบริษัท พร้อม conversation history

- Hint: ใช้ list เก็บ chat_history และ prepend context ทุก turn

ส่วนที่ 8: เอกสารอ้างอิงและแหล่งเรียนรู้

Libraries

- PyMuPDF: <https://pymupdf.readthedocs.io>
- PyThaiNLP: <https://pythainlp.github.io>
- Qdrant: <https://qdrant.tech/documentation>
- Ollama: <https://ollama.ai>

บทความแนะนำ

- "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" — Lewis et al. (2020)
- รายงานทางเทคนิค BGE-M3 (BAAI)
- เอกสาร Qdrant Hybrid Search